

SURVEY

A Survey of Text Deduplication: From Syntactic Matching to Semantic Understanding

JEONGMIN KIM^{ID} AND YEONSU PARK^{ID}

Department of Computer Science and Engineering, Kangwon National University, Chuncheon 24341, Republic of Korea

Corresponding author: Yeonsu Park (yeonsu.park@kangwon.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korean Government (MSIT) under Grant RS-2023-00242528.

ABSTRACT The exponential growth of digital text has made text deduplication, the process of identifying and eliminating redundant information, a critical task for enhancing data quality, optimizing computational resources, and improving the performance of downstream applications, particularly the training of massive AI models. This survey provides a comprehensive and structured overview of the field, beginning with a formal taxonomy of duplicate types including exact, near, and semantic, and a detailed breakdown of the core deduplication pipeline from preprocessing to similarity metrics. We then systematically review the evolution of algorithmic approaches, covering foundational syntactic techniques such as shingling and Locality Sensitive Hashing (LSH), as well as modern semantic methods driven by text embeddings, Transformers, and Large Language Models (LLMs). Beyond algorithms, the survey addresses the critical aspects of system design and scalability for data at the petabyte scale, including architectural patterns, distributed processing, and considerations for dynamic, streaming environments. A detailed examination of diverse applications, rigorous evaluation methodologies, and standard benchmarks provides a practical context for these techniques. Finally, we synthesize the persistent challenges and identify key open research questions, culminating in a visionary outlook on future research directions centered on advanced semantic intelligence, deduplication across multiple modalities and languages, and the engineering of trustworthy, private, and fair systems. This work serves as an essential reference for both newcomers and experienced researchers, providing a complete roadmap of the text deduplication landscape from foundational principles to the frontiers of analysis driven by Artificial Intelligence (AI).

INDEX TERMS Large language models, semantic duplicate detection, text deduplication.

I. INTRODUCTION

The digital era is characterized by an unprecedented explosion of textual data from diverse sources such as the web, social media, scientific publications, and enterprise records. While this abundance offers immense opportunities, it concurrently presents a significant challenge: the proliferation of redundant information. Text deduplication, the systematic process of identifying and eliminating or consolidating such redundant textual data, has thus emerged as a critical enabling technology across numerous domains. Its fundamental goal is to ensure that each distinct piece of information is

represented uniquely, thereby enhancing data quality, optimizing storage and computational resources, and improving the performance of downstream analytical tasks, including the training of large-scale Artificial Intelligence (AI) models.

Effectively addressing textual redundancy requires a nuanced understanding that “duplication” is not a monolithic concept. It spans a spectrum from trivial identity to profound semantic equivalence. This survey categorizes textual duplicates into three primary types, each presenting unique detection challenges and demanding distinct algorithmic approaches:

- **Exact Duplicates:** These are verbatim, character-for-character copies, often arising from data replication

The associate editor coordinating the review of this manuscript and approving it for publication was Sawyer Duane Campbell^{ID}.

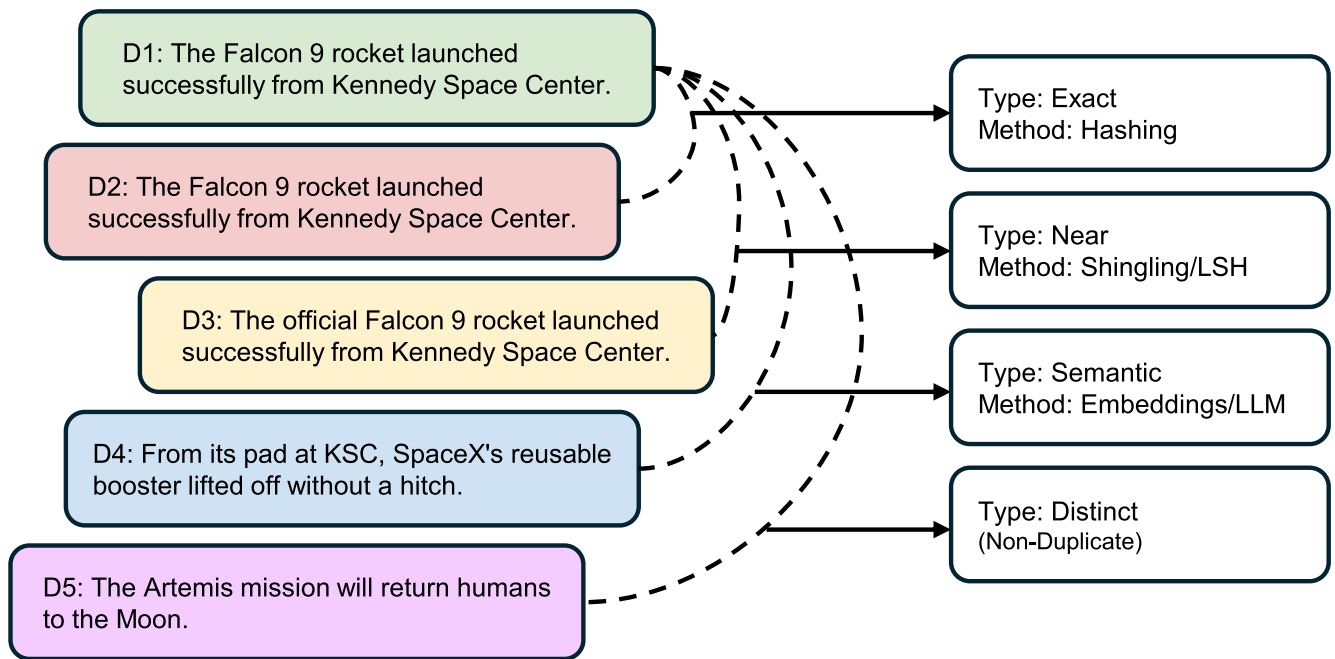


FIGURE 1. The spectrum of duplicate types illustrated with our running example. The documents (D1-D5) serve as a consistent reference throughout this survey to demonstrate how different deduplication techniques operate.

or copy-paste operations. Their detection, typically via cryptographic hashing, is computationally the least complex but crucial for foundational data cleaning.

- **Near-Duplicates:** These texts exhibit minor lexical or syntactic variations (e.g., typos, slight edits, formatting changes) while retaining substantial content overlap. Identifying them is vital for handling noisy real-world data and cleaning web content.
- **Semantic Duplicates:** Representing the most complex challenge, these texts convey the same core meaning or intent despite potentially significant differences in wording and structure (e.g., paraphrases). Their detection necessitates deep language understanding, increasingly powered by advanced embeddings and Large Language Models (LLMs).

To make these concepts concrete, we will use a consistent running example throughout this survey (Figure 1). The choice of deduplication techniques, algorithmic complexity, and evaluation methodologies is heavily influenced by which of these duplicate types is being targeted.

The deluge of data in the current Big Data era, coupled with the voracious data appetite of modern AI systems, particularly LLMs, has elevated text deduplication from a mere data cleaning step to a strategic imperative. Failure to manage redundancy effectively leads to inflated storage costs, severely hampered computational efficiency in data processing pipelines, and critically, compromised performance and reliability of AI models. Redundant training data can cause LLMs to overfit, memorize specific instances rather than learn generalizable patterns, exhibit “multi-epoch

degradation” [1], and even amplify existing data biases. Conversely, meticulous deduplication of training corpora has been shown to yield substantial quantitative benefits [2], [3]. For instance, Lee et al. [2] demonstrated that deduplicating training data reduces the rate of emitting memorized sequences by a factor of 10, significantly mitigating privacy risks. Furthermore, semantic deduplication strategies, such as SemDeDup [4], have been shown to remove up to 50% of a subset of LAION dataset with minimal performance loss, effectively doubling the training speed. Similarly, in the domain of code generation, Kocetkov et al. [5] reported that near-deduplication of The Stack dataset significantly boosted performance across all experiments, outperforming models trained on non-deduplicated data. Similar benefits extend to information retrieval, where removing redundant documents enhances search relevance and user experience, and to overall data quality management, fostering trust in data-driven insights. Text deduplication, given its critical importance, finds diverse applications across numerous fields, including web search engine indexing, large-scale AI/ML dataset curation [2], [6], academic plagiarism detection [7], news aggregation, software engineering, clinical information management [8], knowledge base construction, and legal e-discovery.

Despite the critical importance of text deduplication across these diverse applications, existing literature lacks a comprehensive synthesis that systematically addresses the full spectrum of approaches from syntactic matching to semantic understanding. While individual techniques and domain-specific applications have been extensively

studied [9], [10], [11], [12], [13], [14], no unified survey exists that bridges traditional lexical methods with modern deep learning approaches, particularly in the context of large-scale AI development where deduplication has evolved from a preprocessing step to a strategic imperative. This survey offers a uniquely comprehensive and contemporary perspective by:

- 1) Synthesizing foundational principles with cutting-edge advancements, particularly focusing on the profound impact of LLMs on semantic deduplication.
- 2) Providing an in-depth analysis of system design and scalability considerations for petabyte-scale data, alongside algorithmic discussions.
- 3) Holistically addressing the entire lifecycle, from core concepts to diverse applications, persistent challenges, and synergistic interdisciplinary connections.
- 4) Identifying key open research questions and future directions aimed at guiding subsequent research towards more robust, scalable, interpretable, and responsible deduplication systems.

To construct this survey, we employed a systematic literature review methodology. Our search spanned major academic databases, including IEEE Xplore, the ACM Digital Library, and arXiv, focusing on publications from the last two decades to capture both foundational and recent advancements. The search query combined primary keywords such as “text deduplication,” “near-duplicate detection,” and “semantic duplicate detection” with secondary terms including “locality sensitive hashing,” “text embeddings,” “large language models,” and “data cleaning.” We prioritized peer-reviewed journal articles, conference proceedings, and highly-cited technical reports. The selection criteria favored works that introduced novel algorithms, presented scalable system architectures, provided rigorous empirical evaluations, or offered insightful theoretical analyses. The collected literature was then categorized according to the taxonomy presented in this paper—covering duplicate types, algorithmic approaches, system design, and applications—to synthesize a structured and coherent narrative of the field’s evolution and current state.

This survey is structured to guide the reader through this multifaceted landscape. Section II establishes core concepts and problem formulations. Section III details foundational syntactic and lexical methods. Section IV explores modern semantic, embedding, and learning-based techniques. Section V addresses system design for large-scale implementation. Section VI showcases diverse applications. Section VII outlines persistent challenges and future research directions. Finally, Section VIII concludes the survey.

II. CORE CONCEPTS, PROBLEM FORMULATION, AND PRELIMINARIES

A rigorous and systematic approach to text deduplication requires a clear formalization of its core concepts, a precise articulation of the problem, and a foundational understanding

of the preliminary processing steps involved. This section establishes this groundwork, beginning with a formal taxonomy of duplicate types, which is essential for framing the problem and selecting appropriate algorithms. It then outlines the standard problem formulations and the sequential pipeline of text preprocessing, representation, and similarity measurement that underpins nearly all deduplication systems. The section concludes by examining the theoretical and computational complexities inherent to the task, which motivate the development of the practical, scalable algorithms discussed throughout this survey.

A. FORMALIZING DUPLICATE TYPES

The first step toward a principled approach to text deduplication is establishing a formal taxonomy of duplicate types, as the nature of the targeted redundancy dictates the required algorithmic sophistication. We formally define three primary categories of duplicates—exact, near, and semantic—each representing an increasing level of abstraction and detection complexity. These definitions provide a rigorous framework for understanding the scope and challenges associated with different forms of textual redundancy.

Definition 1 (Exact Duplicates): Let d_i and d_j be two text documents. They are exact duplicates, denoted $d_i \equiv_E d_j$, if their canonical byte sequences are identical. This represents the most direct form of redundancy, where texts are verbatim copies [2].

Definition 2 (Near-Duplicates): Let $S(d_i, d_j)$ be a lexical or structural similarity function. Given a similarity threshold $\tau \in [0, 1]$, d_i and d_j are near-duplicates, denoted $d_i \equiv_N d_j$, if $S(d_i, d_j) \geq \tau$. This category encompasses texts with minor variations that do not substantially alter the core information content [17].

Definition 3 (Semantic Duplicates): Let $M(d)$ be a function that maps a document to its underlying meaning or intent. Two documents d_i and d_j are semantic duplicates, denoted $d_i \equiv_S d_j$, if $M(d_i) = M(d_j)$. This is the most nuanced category, covering texts that are conceptually equivalent despite significant differences in wording and structure [4].

Table 1 provides a consolidated overview of these duplicate types, summarizing their key characteristics, common causes, and typical detection approaches. This taxonomy is not merely academic; it is a practical necessity that directly informs the selection of appropriate algorithms and the design of system architectures. The distinctions outlined here form the basis for structuring the deduplication task, which is intrinsically linked to the specific problem formulation adopted and the overarching goals of the deduplication effort.

B. PROBLEM FORMULATION

The text deduplication challenge can be fundamentally framed within the broader context of text classification. While Siino et al. [23] investigated the impact of preprocessing across various modern and traditional text classification architectures, their findings on model sensitivity

TABLE 1. Summary of duplicate types, characteristics, causes, and detection approaches.

Duplicate Type	Key Characteristics	Common Causes	Typical Detection Approach
Exact	Identical at the character or byte level. No ambiguity in identification. Computationally least intensive to detect.	Data replication, content mirroring, copy-paste operations [15], boilerplate text, repeated log entries. Prevalent in large corpora [2].	Cryptographic hashing (e.g., Secure Hash Algorithm 256-bit (SHA-256), MD5). Identical texts produce identical hash values, enabling fast comparison [16].
Near	High lexical/structural similarity with minor variations. The boundary is defined by a similarity threshold. Captures subtle changes without flagging distinct texts.	Typographical errors, minor edits/revisions, formatting changes (e.g., whitespace, HTML), template-based generation [8], Optical Character Recognition (OCR) errors [17].	Fingerprinting (e.g., MinHash, SimHash) with Locality Sensitive Hashing (LSH) [18]. Set-based metrics (e.g., Jaccard) on n-grams/shingles [19]. String metrics (e.g., edit distance).
Semantic	Convey the same meaning or intent with different wording/structure (paraphrasing). Low lexical overlap but high conceptual similarity. Detection requires deep language understanding.	Intentional rewriting [20], summarization of the same source, different phrasings of the same answer or fact [21].	Semantic text embeddings (e.g., from Bidirectional Encoder Representations from Transformers (BERT), LLMs) with cosine similarity [4]. Supervised models (e.g., Siamese networks). Logical entailment checks [22].

and performance are relevant to deduplication tasks as well. In this view, duplicate detection can be adapted as a binary classification task where a model f maps a pair of documents (d_i, d_j) to a label $y \in \{0, 1\}$, representing ‘distinct’ or ‘duplicate’. This formulation allows deduplication to leverage the extensive advancements in classification architectures analyzed by Siino et al., from traditional feature-based models to modern Transformer-based approaches. The most common formulation is the all-pairs comparison problem, which seeks to identify all pairs of documents (d_i, d_j) in a collection that satisfy a given duplicate definition (e.g., $d_i \equiv_N d_j$). While theoretically comprehensive, its naive $\mathcal{O}(N^2)$ complexity makes it impractical for large datasets, motivating the development of scalable approximation algorithms. A related formulation is record linkage (or entity resolution), which aims to identify records across one or more datasets that refer to the same real-world entity, often relying on textual similarity of key attributes [14]. Finally, deduplication can be framed as a clustering task, where the goal is to group all duplicate documents into clusters, with each cluster representing a single unique concept or piece of information.

These technical formulations are motivated by several ultimate goals, which define the final output and practical utility of the deduplication process. A primary objective is often canonicalization, which involves selecting a single, authoritative representative from each duplicate cluster to be retained, while other copies are either discarded or linked to it [2]. This directly enables the goal of redundancy removal, which is critical for reducing storage costs, improving computational efficiency, and enhancing the quality of machine learning training data by preventing model degradation and overfitting [1], [3], [24]. In other contexts, such as plagiarism analysis or version tracking, the goal is not removal but linking, where all duplicate instances are identified and interconnected to analyze patterns of textual reuse. The choice of problem formulation and ultimate goal is thus a

strategic decision that must align with the targeted duplicate types and navigate the inherent trade-offs between accuracy and computational resources.

C. THE DEDUPLICATION PIPELINE I: TEXT PREPROCESSING

Effective duplicate detection begins with a crucial preprocessing pipeline that transforms raw, often noisy, text into a clean, normalized format suitable for algorithmic comparison. Standard preprocessing involves text cleaning, normalization, and tokenization [25], [26], [27], [28], [29], [30]. While often treated as a static step, recent research by Siino et al. [31] highlights that preprocessing choices significantly impact model performance, even for modern Transformers. Their comparative survey reveals that an appropriate preprocessing strategy can improve classification accuracy by up to 25% (e.g., XLNet on IMDB dataset). Conversely, improper preprocessing can degrade the performance of sophisticated models to the point where they are outperformed by simple classifiers like Naïve Bayes. Thus, preprocessing must be dynamically adapted to the specific model architecture and downstream task. The specific combination of techniques is tailored to the dataset’s characteristics and targeted duplicate types, forming the basis for subsequent representation strategies.

D. THE DEDUPLICATION PIPELINE II: TEXT REPRESENTATION

Following preprocessing, the normalized text must be converted into a structured representation amenable to similarity computation, a step that fundamentally shapes the nature of the subsequent comparison. This second stage of the pipeline transforms text into formats that capture lexical, statistical, or structural properties. Documents are represented using established techniques (character sequences [32], Bag-of-Words [33], Term Frequency-Inverse Document Frequency (TF-IDF) [34], n-grams [35]) as appropriate for the chosen

similarity metrics. These representations, ranging from simple sets of words to weighted n-gram profiles, provide the structured input upon which various similarity and dissimilarity metrics can be applied to quantify the degree of redundancy.

E. THE DEDUPLICATION PIPELINE III: SIMILARITY AND DISSIMILARITY METRICS

With text structured into a chosen representation, the final step in the core comparison pipeline is to apply a quantitative metric to measure the similarity or dissimilarity between pairs of documents. The choice of metric is intrinsically linked to the text representation and the targeted duplicate type. String-based metrics, such as Levenshtein distance or Jaro-Winkler distance [36], [37], operate on character sequences and are effective for near-duplicates with typographical errors. Set-based metrics operate on representations like sets of words or n-grams; the Jaccard Index, which measures the intersection over the union of two sets, is a standard for near-duplicate detection [35], alongside the Sørensen-Dice [38] and Overlap coefficients [39]. Vector-space metrics are applied to vector representations like TF-IDF or advanced embeddings; Cosine Similarity, which measures the angle between two vectors and is insensitive to document length, is a cornerstone for assessing content similarity [33], while Euclidean distance measures the magnitude of difference. The selection of an appropriate metric is thus a critical design choice that directly influences both the accuracy and the computational cost of the deduplication process.

F. THEORETICAL UNDERPINNINGS AND COMPUTATIONAL COMPLEXITY

The practical implementation of the deduplication pipeline is governed by fundamental theoretical and computational constraints that motivate the design of scalable algorithms. The most significant theoretical hurdle is the computational complexity of identifying all duplicate pairs within a large collection, which can be formalized as follows:

Proposition 1 (Complexity of Exhaustive Comparison):

Given a collection of N documents, identifying all duplicate pairs by comparing every document with every other document requires $N(N - 1)/2$ comparisons. This results in a time complexity of $\mathcal{O}(N^2 \cdot C_{sim})$, where C_{sim} is the cost of a single similarity comparison between two documents.

This quadratic complexity renders naive all-pairs comparison computationally infeasible for the massive datasets common today, such as those used in training large language models [2] or managing clinical records [15]. This bottleneck necessitates the development of sophisticated candidate reduction strategies (e.g., blocking, Locality-Sensitive Hashing (LSH)) to approximate an exhaustive search efficiently. The cost is further compounded by the complexity of the similarity function C_{sim} itself, which can range from efficient hash comparisons to expensive string distance calculations

(e.g., $\mathcal{O}(mn)$ for Levenshtein) or deep learning model inferences.

A deeper theoretical challenge lies in the inherent limits of distinguishability for different duplicate types. For exact duplicates, the problem is theoretically simple, as cryptographic hashes provide a computationally efficient and highly reliable method for verifying identity. The challenge for near-duplicates is the ambiguity in defining “nearness,” which relies on empirically-tuned thresholds for similarity metrics [14]. The most profound theoretical difficulties arise with semantic duplicates, as formalizing “same meaning” is an open problem in natural language understanding [40], [41]. Detecting semantic equivalence pushes the limits of AI, requiring models to perform complex reasoning and leverage world knowledge, where even advanced models struggle to distinguish true equivalence from mere topical relatedness [42]. These theoretical constraints, the combinatorial explosion of comparisons and the escalating difficulty of defining similarity, are the primary drivers for the algorithmic innovations explored in the subsequent sections of this survey.

III. FOUNDATIONAL ALGORITHMIC APPROACHES: SYNTACTIC AND LEXICAL METHODS

Foundational approaches to text deduplication operate on the syntactic and lexical properties of documents, forming a computationally efficient bedrock for identifying exact and near-duplicates. These methods analyze the surface-level characteristics of text—such as character sequences and token distributions—rather than its deeper semantic meaning. This section presents a systematic taxonomy of these techniques, progressing from the simplest case of exact matching to scalable strategies for near-duplicate detection.

This pipeline, visualized in Figure 2, begins with cryptographic hashing for verbatim copies. For texts with minor variations, it proceeds through three key stages: (1) extracting robust lexical features via shingling, (2) compressing these features into compact fingerprints using methods like MinHash or SimHash, and (3) employing LSH for efficient, large-scale candidate pair generation. The following subsections detail each component of this foundational pipeline, illustrating how they collectively enable scalable and effective deduplication based on textual form.

Figure 3 provides a concrete walkthrough of this syntactic pipeline using our near-duplicate running example. In Step ①, we start with the original document D1 and a modified version, D3, which contains a single additional word (“official”). This scenario represents a common cause of near-duplicates. Step ② (Normalization) is crucial for eliminating superficial variations like capitalization and punctuation, resulting in clean, lowercase strings. The core feature extraction occurs in Step ③, where each normalized string is decomposed into a set of 3-gram shingles (i.e., sequences of three consecutive words). Finally, Step ④

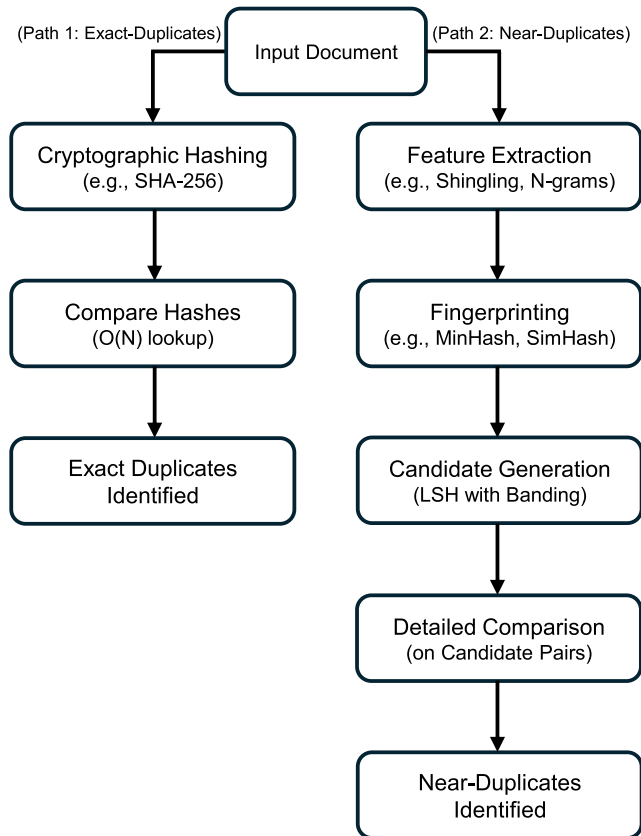


FIGURE 2. A conceptual pipeline for foundational syntactic and lexical deduplication methods. The process bifurcates to handle exact duplicates via direct hashing and near-duplicates through a multi-stage process of feature extraction, fingerprinting, and scalable candidate generation.

demonstrates how the Jaccard Index is calculated. By comparing the two sets of shingles, we obtain a Jaccard score of 0.7, which is typically above the threshold for detecting near-duplicates. This entire process illustrates the method's core strength: its resilience to minor lexical and syntactic noise.

A. EXACT MATCHING VIA CRYPTOGRAPHIC HASHING

The most direct method for identifying verbatim duplicates is exact matching, which is most efficiently implemented using cryptographic hashing. Cryptographic hash functions like SHA-256 convert texts into fixed-length signatures for $\mathcal{O}(1)$ exact duplicate detection. To perform deduplication, each document is hashed, and the resulting signature is stored in a fast lookup structure, such as a hash set. When a new document is processed, its hash is computed and checked for existence in the set; a match indicates an exact duplicate with extremely high probability. The primary advantage of this approach is its exceptional scalability; by transforming expensive string comparisons into fixed-size hash comparisons, the task of finding all exact duplicates in a corpus of size N is reduced from a computationally prohibitive $\mathcal{O}(N^2)$ complexity to a highly efficient, near-linear $\mathcal{O}(N)$ process. This efficiency is vital for preprocessing massive datasets for training LLMs, where removing exact repetitions mitigates model memorization and reduces training costs [2],

Step ① Input Documents (Near-Duplicates)

D1: The Falcon 9 rocket launched successfully from Kennedy Space Center.
D3: The official Falcon 9 rocket launched successfully from Kennedy Space Center.

Step ② Normalization (lowercase, remove punctuation)

D1': the falcon 9 rocket launched successfully from kennedy space center
D3': the official falcon 9 rocket launched successfully from kennedy space center

Step ③ Feature Extraction (3-gram Shingling)

Shingles(D1'): {the falcon 9, falcon 9 rocket, 9 rocket launched, rocket launched successfully, launched successfully from, ...} → Set S1

Shingles(D3'): {the official falcon, official falcon 9, falcon 9 rocket, 9 rocket launched, rocket launched successfully, ...} → Set S2

Step ④ Similarity Calculation (Jaccard Index)

$$J(S1, S2) = \frac{|S1 \cap S2|}{|S1 \cup S2|}$$

Result: High Jaccard Score (0.7) → Near-Duplicate

FIGURE 3. An illustration of the syntactic near-duplicate detection pipeline. The process, using 3-gram shingling, correctly identifies the pair of documents as a near-duplicate with a jaccard score of 0.7.

[3], [6]. However, the sensitivity of cryptographic hashes to any modification renders them incapable of detecting near-duplicates, thus motivating the more nuanced, multi-stage methods required for texts with minor variations. For instance, documents D1 and D2 from our running example would produce identical SHA-256 hashes.

B. FEATURE EXTRACTION FOR NEAR-DUPLICATE DETECTION

To detect near-duplicates that elude exact hashing, documents must first be transformed into sets of robust lexical features, a process commonly achieved through shingling. This

technique canonicalizes a document (e.g., by lowercasing and removing punctuation) and then decomposes it into a set of overlapping, contiguous sequences of tokens, known as shingles (typically word n -grams of length 5 to 10). The resulting set of shingles serves as a document's representative feature set, which is inherently resilient to minor document alterations such as the reordering of sentences or small insertions and deletions, as a substantial portion of the shingles will remain unchanged [35]. As demonstrated with documents D1 and D3 in Figure 3, this process is inherently resilient to minor alterations. The similarity between two documents can then be quantified by comparing their shingle sets using a set-based metric, most commonly the Jaccard Index, which measures the ratio of the size of the intersection to the size of the union of the two sets [43]. This feature extraction stage is fundamental to many large-scale deduplication pipelines, including those used to curate web-scale corpora and improve language model training data by removing near-duplicate content [2]. While comparing these feature sets directly is effective for assessing similarity, the computational cost of performing pairwise Jaccard calculations for all document pairs remains a significant bottleneck, necessitating the use of compression techniques like fingerprinting to enable efficient estimation at scale.

C. EFFICIENT SIMILARITY ESTIMATION VIA FINGERPRINTING

To overcome the computational burden of comparing large feature sets, fingerprinting techniques compress these sets into compact, fixed-size signatures, enabling efficient approximate similarity estimation. These methods are pivotal for making near-duplicate detection scalable in large-scale systems [44]. The two most prominent fingerprinting strategies are MinHash and SimHash. MinHash is a probabilistic technique specifically designed to approximate the Jaccard similarity of two sets, making it a natural fit for comparing the shingle sets generated in the previous stage. Its core principle is that the probability of two sets having the same minimum hash value, after applying a random permutation to all elements, is equal to their Jaccard similarity. In practice, this is achieved by applying multiple hash functions to each shingle in a document's set and retaining only the minimum hash value for each function, thereby creating a compact MinHash signature. SimHash, in contrast, is designed to produce fingerprints where the Hamming distance between them approximates the cosine angle of the documents' original vector representations (e.g., TF-IDF vectors). It works by hashing each weighted feature of a document into a bit-vector, and then combining these vectors into a final fingerprint whose bits are determined by the sign of the summed vector dimensions. Both MinHash and SimHash are instrumental in large-scale deduplication of web data and noisy scanned texts, as they generate compact fingerprints that are resilient to minor variations [45],

[46]. Ultimately, these fingerprints serve as the essential, compressed inputs for scalable indexing frameworks like Locality-Sensitive Hashing, which solve the final problem of avoiding exhaustive all-pairs comparisons.

D. SCALABLE CANDIDATE GENERATION VIA LSH

The final stage in scaling near-duplicate detection is Locality-Sensitive Hashing (LSH), a probabilistic method that uses the generated fingerprints to efficiently identify candidate pairs without performing $\mathcal{O}(N^2)$ comparisons. LSH overcomes the brute-force complexity by hashing similar items into the same "bucket" with a higher probability than dissimilar items, thereby drastically pruning the search space to only those pairs most likely to be duplicates. This principle is formalized in the definition of an LSH family.

Definition 4 (LSH Family): A family $\mathcal{H}$ of hash functions is said to be (d_1, d_2, p_1, p_2) -sensitive for a distance metric D if for any two points x, y and a function h chosen uniformly at random from $\mathcal{H}$:

- 1) if $D(x, y) \leq d_1$, then $\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \geq p_1$.
- 2) if $D(x, y) \geq d_2$, then $\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq p_2$.

For LSH to be effective, it must hold that $d_1 < d_2$ and $p_1 > p_2$.

The fingerprinting methods discussed previously are themselves LSH families: MinHash is sensitive to Jaccard distance, while SimHash is sensitive to cosine distance (via Hamming distance on the fingerprints). In practice, the precision-recall trade-off of LSH is tuned using the banding technique, which combines multiple hash values into bands. Two documents are considered a candidate pair if they match on at least one full band, an approach that amplifies the probability of collision for similar items while keeping it low for dissimilar ones. By restricting expensive, detailed similarity calculations only to pairs that collide in the same LSH bucket, this method makes near-duplicate detection feasible for massive corpora, such as those used in clinical informatics [8] or for pre-training LLMs [2]. LSH is thus a cornerstone of modern scalable deduplication, forming the critical candidate generation step in systems ranging from large-scale entity resolution [47] to interactive data cleaning frameworks [48].

IV. MODERN TECHNIQUES: SEMANTIC, EMBEDDING, AND LEARNING-BASED METHODS

The foundational lexical and syntactic methods discussed in Section III, while effective for exact and near-duplicate detection, are fundamentally limited by their reliance on surface-level text features. To identify semantic duplicates, texts that convey the same meaning using different vocabulary and structure, a deeper understanding of language is required. This necessity catalyzed a shift towards modern learning-based techniques that aim to capture the underlying meaning of text. This section describes the evolution of these methods, beginning with the development of semantic text embeddings, progressing through the architectural revolution

of Transformers, and culminating in the advanced capabilities of LLMs. This progression marks a clear trajectory from representing words to understanding world knowledge, fundamentally reshaping the possibilities for text deduplication.

A. THE RISE OF SEMANTIC EMBEDDINGS: FROM STATIC TO CONTEXTUAL REPRESENTATIONS

The initial leap beyond lexical matching involved representing text within a continuous vector space, where semantic similarity corresponds to geometric proximity. This approach, formalized through text embeddings, transforms textual units like words or sentences into dense numerical vectors, enabling the quantitative measurement of semantic relatedness using metrics like cosine similarity. This vector-space paradigm provides a powerful, mathematically grounded framework for moving beyond simple keyword overlap to a more nuanced comparison of meaning, forming the bedrock of modern semantic deduplication.

Figure 4 provides a concrete walkthrough of the semantic deduplication process. In Step ①, we start with our three running example documents. In Step ②, the `all-mpnet-base-v2` model [49] maps each document into a 768-dimensional vector. The results are presented in Step ③. As detailed in the caption, the final similarity scores are calculated using the entire high-dimensional vectors, confirming the semantic relationships quantitatively. The plot offers a simplified 2D projection of this high-dimensional space, effectively illustrating the core concept: semantically similar vectors occupy a closer region in the vector space. This demonstrates how modern embeddings capture meaning beyond simple keyword overlap.

Early breakthroughs in semantic representation emerged with Word2Vec [50] and GloVe [51]. These models learn fixed vectors for each word based on co-occurrence patterns, enabling semantic operations like analogies.

Static embeddings suffer from critical context-insensitivity. A single vector cannot disambiguate polysemous words or handle out-of-vocabulary terms [52], severely limiting their utility for sentence-level comparison.

To overcome the context-insensitivity of static vectors, a new class of models emerged that generate contextualized embeddings, where a word's representation is dynamically computed based on its surrounding sequence. Pioneering models like Embeddings from Language Models (ELMo) [52], typically leveraging recurrent architectures such as bidirectional Long Short-Term Memory networks (LSTMs) [53], produce different vectors for the same word depending on its usage. This allows the model to distinguish between different senses of a word, providing a much richer and more accurate semantic representation at the word level [52]. By considering the broader context, these models can better capture the intended meaning of short texts [21] and lay the essential groundwork for more sophisticated sentence and document-level understanding. This shift from static to contextualized embeddings marked a pivotal advance,

enabling a more nuanced assessment of semantic similarity that is crucial for effective deduplication.

B. THE TRANSFORMER REVOLUTION: FROM SEQUENTIAL PROCESSING TO GLOBAL CONTEXT

While Recurrent Neural Network (RNN)-based models [53] introduced context, their inherently sequential nature, processing text one token at a time, created computational bottlenecks and struggled to capture long-range dependencies effectively. The Transformer architecture instigated a revolution by replacing sequential recurrence with a parallelizable self-attention mechanism. Self-attention allows a model to weigh the importance of all other words in a sequence when encoding a given word, regardless of their distance. This ability to model global context directly and efficiently overcame the primary limitations of RNNs, paving the way for deeper, more powerful, and highly scalable Pre-trained Language Models (PLMs) that have redefined semantic understanding [54].

This new architecture enabled the highly successful pre-train and fine-tune paradigm, where models like BERT [55], RoBERTa [56], and Decoding-enhanced BERT with disentangled attention (DeBERTa) [57] first learn general-purpose language representations from massive unlabeled text corpora [26]. Through self-supervised objectives like Masked Language Modeling (MLM) [55], these PLMs acquire a robust grasp of syntax, semantics, and world knowledge [55]. This general competence is then adapted for specific deduplication-related tasks, such as semantic textual similarity (STS) or paraphrase identification, by fine-tuning the model on smaller, task-specific labeled datasets. This approach has proven highly effective, with Transformer-based models achieving state-of-the-art results in specialized domains like bug report triaging [58], scientific document analysis [59], and cybersecurity [60], showcasing their adaptability and superior ability to capture subtle semantic patterns.

To effectively adapt PLMs for similarity tasks, specialized fine-tuning architectures such as Siamese and Triplet networks are commonly employed to create a semantically meaningful embedding space. A Siamese network uses two identical, weight-sharing encoders (e.g., two BERT models) to process a pair of texts, with a contrastive loss function that minimizes the distance between vectors of similar texts while maximizing it for dissimilar ones. Triplet networks refine this by learning from an (anchor, positive, negative) triplet, ensuring the anchor is closer to the positive than the negative by a defined margin, which often yields more robust similarity distinctions. Sentence-BERT (SBERT) is a prominent example that uses such a structure to produce high-quality sentence embeddings from PLMs [49]. These interaction-focused training schemes are pivotal for generating embeddings where vector proximity accurately reflects semantic similarity, enabling efficient and precise large-scale comparisons for deduplication [18].

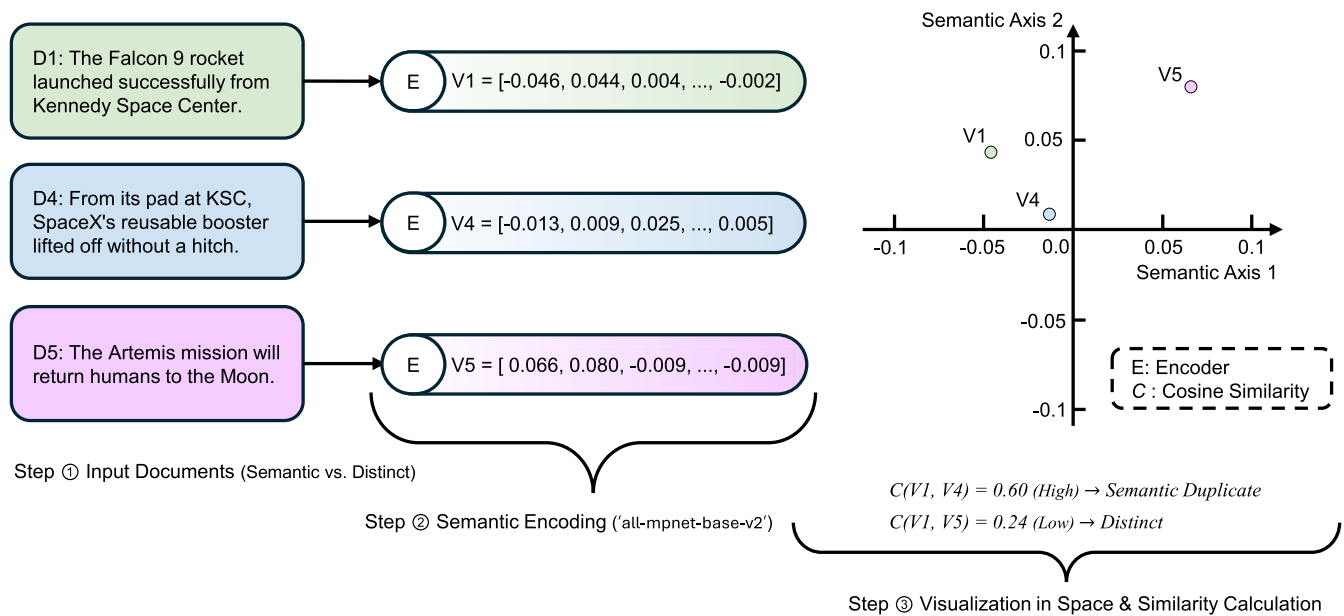


FIGURE 4. Documents are mapped to a high-dimensional vector space using the 'all-mpnet-base-v2' sentence embedding model, where proximity indicates semantic similarity, then visualized here by plotting the first two dimensions of their embeddings to highlight these relationships. The visual clustering is quantitatively supported by cosine similarity scores calculated on the full 768-dimensional vectors: 0.60 for the near-duplicates (V1, V4) versus 0.24 for the distinct pair (V1, V5).

C. LARGE LANGUAGE MODELS: EMERGENT ABILITIES AND ZERO-SHOT DEDUPLICATION

The continued scaling of Transformer architectures in parameter count and training data volume has given rise to LLMs like the Generative Pre-trained Transformer (GPT) series [61]. As comprehensively surveyed by Mienye et al. [62], these foundational architectures have evolved from task-specific models to versatile systems capable of complex reasoning. We posit that such emergent capabilities provide the necessary cognitive depth to further advance semantic deduplication. Building upon the foundation of PLMs, LLMs demonstrate an unparalleled ability to understand subtle paraphrases, complex analogies, and implied meanings, often leveraging a vast internal repository of world knowledge acquired during their extensive training [63]. This deep, context-aware understanding allows them to identify semantic equivalences that share minimal lexical overlap, moving beyond pattern recognition towards a more human-like comprehension of text [64]. These emergent properties enable new, more flexible paradigms for deduplication that do not strictly require task-specific fine-tuning.

A key advantage of LLMs is their capacity for zero-shot or few-shot similarity assessment via natural language prompting. Instead of fine-tuning on a large labeled dataset, one can simply instruct the model to determine if two texts are duplicates (e.g., "Do the following two sentences convey the same meaning?"). For instance, an LLM could be prompted to directly compare D1 and D4, determining their semantic equivalence without any task-specific fine-tuning.

This approach is exceptionally powerful for specialized deduplication tasks where labeled data is scarce or expensive to create, offering unprecedented flexibility for practitioners to define and detect duplicates in novel contexts with minimal overhead [63]. While powerful, this approach is not without its challenges, as the quality of the output can be sensitive to the prompt's phrasing and the model's inherent limitations.

Beyond direct prompting, LLMs serve as state-of-the-art text encoders, producing embeddings that capture richer and more nuanced semantic information than smaller PLMs. The hidden-state vectors from LLMs provide superior representations for downstream similarity calculations and have demonstrated exceptional performance in tasks like semantic clustering, which is directly applicable to grouping duplicate documents [65]. Although generating these embeddings is computationally expensive, techniques like knowledge distillation can transfer the representational power of a large "teacher" LLM to a smaller, more efficient "student" model. This highlights a synergistic cycle: effective deduplication is crucial for training high-quality LLMs [2], [4], which in turn become powerful tools for performing even more sophisticated deduplication, pushing the frontier of semantic understanding [4].

D. PROMPT ENGINEERING STRATEGIES FOR DEDUPLICATION

Beyond standard zero-shot prompting, advanced prompt engineering strategies have emerged as powerful tools for semantic deduplication. Wei et al. [66] introduced Chain-of-Thought (CoT) prompting, demonstrating that generating

intermediate reasoning steps significantly improves model performance on complex arithmetic and symbolic tasks. When adapted to the context of deduplication, this paradigm allows the model to explicitly articulate *why* two documents are duplicates (e.g., “Document A mentions X, Document B mentions Y, which implies X...”) before making a final decision.

Furthermore, Chia et al. [67] proposed Contrastive Chain-of-Thought, which utilizes both valid and invalid reasoning demonstrations in the few-shot template. While originally validated on arithmetic and factual reasoning benchmarks, this methodology offers a compelling framework for deduplication. By adapting this approach to present the model with examples of correct duplicate identification alongside incorrect reasoning paths (e.g., falsely flagging distinct texts as duplicates due to lexical overlap), researchers can significantly enhance the model’s ability to discern subtle semantic differences and improve robustness against adversarial non-duplicates.

Additionally, instruction-tuned models such as Instruct-GPT [68] provide a robust foundation for prompt-based deduplication. While the original study focused on general alignment, their explicit optimization to follow natural language instructions allows them to adhere to complex, user-defined matching criteria with greater fidelity than base language models. This instruction-following capability is essential for ensuring that the model strictly observes the specific nuances of a “duplicate” definition provided in the prompt.

E. SYNTHESIS AND COMPARATIVE ANALYSIS

The evolution from static embeddings to large language models presents a spectrum of techniques for semantic deduplication, each with distinct innovations, limitations, and ideal use cases. This progression reflects a trade-off between computational complexity and the depth of semantic understanding, from context-agnostic word vectors to context-aware models capable of complex reasoning. Table 2 synthesizes these characteristics to provide a comparative overview for researchers and practitioners. This summary clarifies the trade-offs inherent in selecting a modern deduplication method, balancing factors like contextual understanding, computational cost, and the need for labeled data.

V. SYSTEM DESIGN, ARCHITECTURE, AND SCALABILITY FOR LARGE-SCALE DEDUPLICATION

Transitioning from algorithmic principles to practical implementation, this section addresses the system-level challenges of building robust, scalable text deduplication pipelines. Successfully processing modern, massive datasets—whether for training large language models [2], [4], curating web-scale corpora [17], or managing extensive clinical records [8], [15]—demands a principled architectural approach. The primary challenges revolve around overcoming prohibitive computational complexity, managing immense data volumes,

optimizing core operations, and handling dynamic data streams. This section systematically deconstructs these challenges, presenting a blueprint for scalable system design that synthesizes architectural patterns, data structures, distributed paradigms, and strategies for managing data dynamics.

A. ARCHITECTURAL BLUEPRINT FOR SCALABLE DEDUPLICATION

A robust and scalable text deduplication system is fundamentally built upon a modular, multi-stage pipeline architecture designed to systematically reduce data complexity and volume. This architecture provides a structured framework for applying various algorithms and optimizations, ensuring that computationally expensive operations are performed on the smallest possible subset of data. The design must address the core trade-offs between accuracy, latency, and resource consumption, which are dictated by the specific application’s requirements and the nature of the data flow. Historical approaches to software availability, such as dynamic binding in web services proposed by Oyeboode and Esenogho [70], highlight the importance of real-time switching mechanisms to maintain system reliability—a principle that remains critical for modern scalable deduplication pipelines.

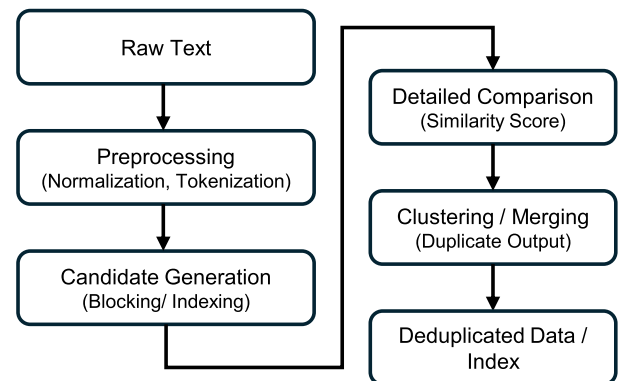


FIGURE 5. A generic, modular pipeline for scalable text deduplication. Each stage is designed to progressively filter and refine the data, reducing the computational burden on subsequent, more intensive stages.

As illustrated in Figure 5, we can formalize this common architectural pattern as a sequence of transformations. A generic deduplication pipeline consists of four primary stages:

- 1) **Preprocessing:** Raw text is transformed into a canonical format suitable for comparison, involving steps like normalization, cleaning, and tokenization (as detailed in Section II-C).
- 2) **Candidate Generation:** The core scalability mechanism, this stage uses efficient blocking or indexing techniques to drastically prune the $\mathcal{O}(N^2)$ search space, identifying a small subset of potential duplicate pairs.
- 3) **Detailed Comparison:** The generated candidate pairs are subjected to more computationally intensive similarity scoring using appropriate metrics (e.g.,

TABLE 2. Comparative analysis of modern semantic deduplication techniques.

Model Family	Core Innovation	Key Limitation	Primary Use Case in Deduplication
Static Embeddings (e.g., Word2Vec, GloVe)	Represents words as dense vectors; captures semantic relationships like synonymy and analogy.	Context-insensitive (polysemy problem); single vector per word; struggles with out-of-vocabulary words.	Baseline semantic duplicate detection where simple word overlap is insufficient; useful for identifying paraphrases with synonym substitutions.
Contextual RNNs (e.g., ELMo, LSTMs)	Generates context-dependent word embeddings, resolving polysemy. Models sequential information.	Sequential processing is slow and not easily parallelizable; struggles with very long-range dependencies.	Detecting semantic duplicates at the sentence level where local context is critical for disambiguation.
Pre-trained Transformers (e.g., BERT, SBERT)	Self-attention mechanism captures global context in parallel; enables deep, powerful pre-trained models.	Requires significant computational resources for fine-tuning and inference; may need large labeled datasets.	High-accuracy semantic duplicate detection via fine-tuning for STS or paraphrase identification; generating high-quality sentence embeddings.
Large Language Models (e.g., GPT-3/4, LLaMA)	Emergent capabilities from massive scale; strong zero/few-shot reasoning and world knowledge [69].	Extremely high computational cost; potential for factual inconsistency (“hallucination”); performance is sensitive to prompting.	Zero-shot duplicate detection for specialized tasks without labeled data; identifying complex, implied, or abstract semantic equivalences.

Jaccard, cosine, or model-based semantic similarity) to accurately assess their relatedness.

4) **Clustering and Output:** Based on the similarity scores, a final decision is made. Pairs exceeding a threshold are declared duplicates, often grouped into clusters, and a canonical representative may be selected before the final output is generated.

A critical architectural decision is the choice of processing paradigm, which depends on the data’s velocity and the application’s latency requirements (summarized in Table 3). Batch processing is employed for static, large-scale datasets where offline, global analysis is feasible and high accuracy is prioritized, common in curating training corpora [2], [17]. Mini-batch processing offers a compromise, handling data in smaller chunks for very large or evolving datasets where full reprocessing is too costly. Finally, stream processing is essential for dynamic, high-velocity environments requiring near real-time duplicate detection, such as filtering live news feeds. Each paradigm imposes distinct constraints on system design, particularly concerning state management and fault tolerance, thereby shaping the implementation of the generic pipeline.

B. OVERCOMING QUADRATIC COMPLEXITY: CANDIDATE GENERATION STRATEGIES

The primary computational hurdle in any large-scale deduplication system is the $\mathcal{O}(N^2)$ complexity of naive all-pairs comparison, which is computationally infeasible for modern datasets. The candidate generation stage is therefore the most critical architectural component for achieving scalability, as its sole purpose is to intelligently prune this vast search space. By employing efficient filtering techniques, this stage identifies a small, high-probability subset of candidate pairs, allowing the more expensive detailed comparison algorithms to be applied judiciously without a significant loss of recall.

Blocking and indexing are the two principal strategies for candidate generation. Blocking partitions the dataset into smaller, disjoint groups (blocks) based on a derived “blocking key,” with comparisons confined to pairs within the same block. Standard blocking uses a key derived from the text, such as the first few words or a phonetic code of a name [71]. More advanced methods like the Sorted Neighborhood Method (SNM) sort records by a key and compare items within a sliding window, capturing near-matches that might fall into adjacent blocks. Indexing, conversely, creates data structures for rapid retrieval of potential matches. Inverted indexes, which map features (e.g., words, n-grams) to the documents containing them, are commonly used to quickly find documents sharing common features. More specialized structures like suffix trees can identify long shared substrings, which are strong indicators of near-duplication.

These strategies are often augmented by probabilistic techniques like LSH, as detailed in Section III-D. LSH serves as a powerful, data-driven blocking mechanism by hashing similar items to the same bucket with high probability [44]. By generating compact fingerprints (e.g., MinHash, SimHash) and indexing them, LSH-based methods efficiently identify candidate pairs for near-duplicate detection at web-scale. The effectiveness of these candidate generation techniques is fundamental to the entire deduplication pipeline, as they make it computationally tractable to process the massive datasets required for training modern AI systems [2], [72] and managing large-scale data services [6], [73].

C. OPTIMIZING CORE OPERATIONS: DATA STRUCTURES AND ALGORITHMIC ENHANCEMENTS

While candidate generation drastically reduces the number of comparisons, the efficiency of the remaining core operations—such as fingerprint management, similarity calculation, and set comparisons—is still critical for overall

TABLE 3. Comparison of processing paradigms for large-scale deduplication.

Paradigm	Data Model	Latency	Typical Use Case	Key Challenge
Batch	Bounded, static dataset	High	Offline LLM training data curation [2]	Managing massive resource usage
Mini-Batch	Sequential, small batches	Medium	Periodic updates to large archives	Balancing batch size and update frequency
Stream	Unbounded, continuous data	Low	Real-time news feed filtering [17]	State management and low-latency processing

system performance. Optimizing these operations requires the use of specialized data structures and algorithmic enhancements designed to minimize memory consumption and accelerate computation. These optimizations are essential for processing the generated candidates and managing the associated metadata at scale.

To manage memory usage, especially when dealing with billions of fingerprints or feature sets, probabilistic data structures are invaluable. Bloom filters [74] provide a memory-efficient way to test for the prior existence of an item (e.g., a document hash), drastically reducing lookups to more expensive persistent storage by quickly filtering out new items. For estimating feature frequencies to guide similarity weighting or blocking, Count-Min Sketch [75] offers a compact alternative to exact counting. Similarly, HyperLogLog [76] can efficiently estimate the cardinality of unique items in a stream, providing vital statistics for system monitoring without storing all unique elements. Beyond probabilistic structures, using compressed representations for text, features, or vectors is crucial for reducing both memory footprint and I/O overhead [16], [77].

To accelerate core computations, several algorithmic optimizations are employed. For similarity calculations, techniques like early exit (terminating a comparison once a dissimilarity threshold is passed) or using bit-parallel algorithms for edit distance can offer significant speedups. For efficient set intersections, which are fundamental to Jaccard similarity calculations on shingle sets, representing sets as sorted lists and using merge-join algorithms is a standard and effective approach. Finally, for fast fingerprint lookups, which are central to LSH-based methods, fingerprints are stored and indexed in specialized data structures or databases optimized for the specific query type (e.g., Hamming distance for SimHash, bucket lookups for MinHash bands) [78], [79]. The careful selection and implementation of these structures and algorithms are thus integral to building high-performance deduplication systems.

D. SCALING OUT: DISTRIBUTED PROCESSING FOR MASSIVE DATA VOLUMES

To handle data volumes that exceed the capacity of a single machine, such as web-scale corpora [25], [80], distributed processing paradigms are indispensable. These frameworks enable horizontal scaling by partitioning the data and distributing the computational workload across a cluster of machines. This approach is fundamental to making petabyte-scale deduplication feasible, transforming

an intractable single-node problem into a manageable, parallelized task.

The implementation of distributed deduplication typically relies on established big data frameworks like Apache Hadoop MapReduce [81] or, more commonly, Apache Spark [82]. Spark's in-memory processing capabilities are particularly well-suited for the iterative nature of many deduplication algorithms, such as clustering or multi-pass LSH. These frameworks primarily leverage data parallelism, where the dataset is split into partitions and the same operation (e.g., text preprocessing, fingerprint generation) is executed concurrently on each partition across different nodes [81]. This strategy is highly effective for the initial, embarrassingly parallel stages of the deduplication pipeline.

Effective distributed implementation, however, requires addressing several key challenges. Data partitioning must be done intelligently, often guided by blocking keys, to co-locate potential duplicates and minimize expensive data shuffling across the network. Load balancing is crucial to ensure that work is evenly distributed and no single node becomes a bottleneck. Most importantly, communication overhead must be minimized through techniques like maximizing data locality (processing data where it resides), broadcasting smaller data structures (e.g., model parameters, LSH configurations), and using efficient data serialization formats. Finally, fault tolerance is critical in long-running, large-scale jobs. Frameworks achieve this through mechanisms like data replication and task checkpointing, which allow the system to recover from node failures without restarting the entire process from scratch [83]. By mastering these distributed computing principles, it becomes possible to apply deduplication to the massive datasets required for modern AI and data science [2], [8].

E. MANAGING DATA DYNAMICS: INCREMENTAL AND REAL-TIME SYSTEMS

Deduplication systems must often operate not on static datasets but in dynamic environments where data arrives as a continuous stream or is frequently updated. These scenarios demand incremental or real-time processing capabilities, which introduce unique challenges related to state management, latency, and adaptability. The goal is to efficiently identify duplicates against an evolving historical context without the luxury of periodic full reprocessing.

Incremental deduplication is designed for datasets that grow over time. The core principle is to maintain a persistent, indexed representation of all previously seen unique items

(or their fingerprints) and compare new arrivals against this index [84]. A key challenge is managing the state of this index, which can grow to an immense size, potentially slowing down query performance over time. Real-time deduplication addresses high-velocity streams where decisions must be made with minimal latency (e.g., milliseconds). This requires extremely fast algorithms and often involves approximation strategies to meet strict time constraints [85]. For instance, neural models used for semantic matching must be heavily optimized for low-latency inference to be viable in such settings [86].

To manage the unbounded nature of streams and control resource consumption, windowing techniques are commonly employed. Sliding or tumbling windows restrict comparisons to a recent time- or count-based portion of the data, focusing on temporally relevant duplicates at the cost of missing older ones. The choice of window size represents a critical trade-off between completeness and resource usage. Furthermore, both incremental and real-time systems must be robust to concept drift, where the statistical properties or topics within the data stream change over time. This requires adaptive models and indexing strategies that can evolve with the data. Ultimately, designing for dynamic data involves a careful balancing act between accuracy, latency, and resource management, often leveraging approximation strategies to make near-real-time processing feasible in practice [87].

VI. APPLICATIONS OF TEXT DEDUPLICATION ACROSS DIVERSE DOMAINS

The theoretical concepts and algorithmic techniques detailed in previous sections find extensive practical application across a wide spectrum of domains, underscoring text deduplication's evolution from a niche data cleaning task to a fundamental component of modern information systems. The utility of deduplication is realized in any context grappling with large volumes of textual data, where redundancy can degrade performance, inflate costs, and compromise data integrity. To provide a structured overview of this broad impact, we categorize these applications into three thematic areas: (1) Web and Information Management, focusing on the curation of public-facing information and large-scale archives; (2) AI and Machine Learning Development, highlighting its critical role in building robust models and knowledge systems; and (3) Domain-Specific Integrity and Efficiency, examining its tailored use in specialized professional fields. Table 4 presents a comprehensive summary of these applications, detailing their primary goals, dominant duplicate types, and the key techniques employed, thereby serving as a concise reference for the subsequent discussion.

A. WEB AND INFORMATION MANAGEMENT

Text deduplication is a foundational technology for managing the scale and redundancy of public web content and large-scale data archives, where the primary challenge is to

curate unique, high-quality information from a deluge of repetitive data. In web search engine indexing, redundancy in the form of syndicated articles, mirrored sites, and boilerplate content creates two problems: it wastes crawling and storage resources, and it degrades search results by cluttering them with repetitive links [45]. Deduplication, particularly near-duplicate detection using techniques like SimHash and LSH (see Section III-D), is essential for identifying a single canonical version of a document, thereby creating a leaner index and a cleaner, more relevant user experience [93]. Similarly, news aggregation services face a constant stream of articles from numerous sources reporting on the same events, leading to high levels of near-duplicate and semantic redundancy. Here, the goal is not just to eliminate copies but to cluster related stories, allowing users to see a single event's coverage from diverse perspectives rather than a list of nearly identical reports. This requires robust, often real-time, semantic clustering techniques that can withstand the "noise" of journalistic variation [17]. In its most fundamental form, deduplication in data archiving and storage optimization directly addresses the economic and logistical burdens of data volume. By using exact-match hashing and sub-file chunking to eliminate redundant data blocks, these systems drastically reduce storage costs, shorten backup times, and minimize network traffic during replication [16]. In all these applications, deduplication transforms the task of information management from merely storing vast quantities of raw data to curating a high-quality, non-redundant, and more valuable information repository.

B. AI AND MACHINE LEARNING DEVELOPMENT

In the development of modern AI systems, text deduplication has evolved from a simple data cleaning step to a strategic imperative for building robust, efficient, and reliable models. The curation of large-scale datasets for pretraining Large Language Models (LLMs) is arguably the most critical contemporary application, as model performance is deeply tied to data quality [6]. The presence of extensive duplicates in training data leads to several adverse effects: it wastes immense computational resources during training [1], biases models toward memorizing frequent samples rather than learning generalizable patterns [3], and can cause performance degradation [2]. Consequently, rigorous deduplication at the exact, near, and semantic levels is a mandatory step in creating high-quality pretraining corpora like C4, The Stack, and others [2], [6]. This process, often leveraging scalable hashing and semantic embedding techniques (see Section III and IV), directly improves model quality and training efficiency [4], [72]. Beyond model training, deduplication is also central to the construction and refinement of knowledge bases (KBs) and ontologies. These structured resources, which underpin many AI tasks like question answering and semantic search, are often populated by extracting facts from heterogeneous sources, leading to

TABLE 4. A summary of key text deduplication applications across diverse domains.

Application Domain	Primary Goal	Dominant Duplicate Type(s)	Key Techniques Employed
Web and Information Management			
Web Search Indexing	Improve search result quality; reduce index bloat and crawling costs.	Near-duplicates (syndicated content, boilerplate), exact duplicates.	LSH (MinHash, SimHash) [18], [44], shingling, clustering [19].
News Aggregation	Cluster related stories; enhance user experience by reducing repetition.	Near-duplicates (rewording), semantic duplicates (same event, different sources).	Noise-robust neural models [17], semantic embeddings, real-time clustering.
Data Archiving & Storage	Minimize storage costs and backup windows; optimize network bandwidth.	Exact duplicates, near-duplicates (sub-file level, minor revisions).	Cryptographic hashing [16], chunking algorithms, binary diffing.
AI and Machine Learning Development			
LLM Dataset Curation	Improve model generalization; reduce training cost and memorization [2], [72].	Exact, near, and semantic duplicates at a massive scale.	Scalable hashing, LSH, semantic embeddings (SBERT, LLMs) [3], [4].
Knowledge Base Refinement	Entity resolution; ensure factual consistency and integrity.	Near-duplicates (aliases, name variations), semantic duplicates (paraphrased facts).	Record linkage techniques, semantic similarity models, graph-based methods [88].
Domain-Specific Integrity and Efficiency			
Academic Integrity	Detect plagiarism; uphold scholarly originality.	Exact (copy-paste), near-duplicates (patchwriting, moderate paraphrasing).	N-gram overlap, document similarity metrics, specialized math/citation analysis [7], [89].
Software Engineering	Efficiently triage bug reports; consolidate developer effort.	Near-duplicates (similar error descriptions), semantic duplicates (same root cause).	TF-IDF, Best Matching 25 (BM25), Transformer-based embeddings (e.g., DeBERTa) [58], [90].
Clinical Information	Improve data quality for research and patient care; reduce Electronic Health Record (EHR) bloat.	Exact (copy-paste), near-duplicates (template-based notes).	Scalable LSH [15], domain-specific semantic models [8].
CRM & Customer Support	Achieve a unified customer view; streamline support ticket management.	Near-duplicates (name/address variants), semantic duplicates (similar queries).	Edit distance [91], phonetic algorithms, semantic similarity models [92].

redundant entities (e.g., “IBM” vs. “International Business Machines”) and paraphrased facts. Entity resolution and semantic fact deduplication are crucial for ensuring the consistency and integrity of the KB, preventing erroneous inferences and creating a reliable foundation for knowledge-intensive applications [14]. For both dataset curation and knowledge base refinement, deduplication is not merely about reducing data size but is a fundamental process that directly shapes the quality, behavior, and reliability of the resulting AI systems.

C. DOMAIN-SPECIFIC INTEGRITY AND EFFICIENCY

Beyond general web and AI applications, text deduplication is critical for enhancing operational efficiency and ensuring informational integrity in specialized professional fields, where the definition of a “duplicate” and the cost of errors are tightly coupled with domain-specific workflows and standards. We can formalize this concept as follows:

Definition 5 (Domain-Adapted Deduplication): Domain-Adapted Deduplication is a methodological paradigm that involves the systematic customization of deduplication algorithms, similarity metrics, and system architectures to align with a specific professional or scientific domain. This adaptation is necessitated by two factors: (1) the domain’s unique lexical, syntactic, and semantic conventions, which

redefine the notion of a “duplicate,” and (2) its distinct operational goals (e.g., latency, cost) and integrity requirements, which dictate the acceptable trade-offs between precision, recall, and computational efficiency.

In domains where integrity is paramount, such as academia and law, Domain-Adapted Deduplication is indispensable. For academic publishing, it powers plagiarism detection systems that compare submissions against vast repositories to identify verbatim copying and paraphrasing, thereby upholding scholarly originality [7]. The challenge lies in detecting sophisticated reuse, including non-textual elements like mathematical formulas or citation patterns [89].

In other fields, the primary driver is operational efficiency. In software engineering, deduplicating bug reports is crucial for preventing multiple developers from unknowingly working on the same underlying issue described with different terminology [94]. This requires semantic matching to link reports describing the same functional flaw, a task for which Transformer-based models have shown great promise [58]. Similarly, in Customer Relationship Management (CRM), deduplication merges fragmented customer records created from varied inputs (e.g., “Jon Doe” vs. “Jonathan Doe”) and consolidates support tickets describing the same problem, enabling a unified customer view and preventing redundant service efforts [14].

Finally, in safety-critical domains like clinical and biomedical information management, deduplication ensures both integrity and efficiency. Clinical notes are notoriously redundant due to copy-paste practices and template-based documentation [15]. Deduplicating these records is essential for creating a clear and concise patient history for clinical decision support and for curating high-quality, non-redundant datasets for training specialized medical language models, which has been shown to improve training efficiency without harming downstream task performance [8]. In each of these specialized domains, a generic approach to deduplication is insufficient; success requires solutions tailored to the unique nature of the data and the high-stakes context of its use.

VII. OPEN CHALLENGES AND FUTURE RESEARCH DIRECTIONS

As text deduplication matures from a data preprocessing step into a critical component of large-scale information systems and AI model development, the field confronts a new frontier of challenges. While significant progress has been made, pushing the boundaries of what is possible requires tackling persistent and emerging problems in semantic understanding, scalability, real-world applicability, and ethical governance. The future trajectory of text deduplication will be defined by a deeper integration with cutting-edge AI, an expansion into new data frontiers, and a foundational commitment to building trustworthy systems. To provide a structured vision for this future, we synthesize the myriad open challenges and opportunities into three primary research thrusts: (1) advancing semantic intelligence and robustness, (2) scaling to new frontiers of data volume, velocity, and variety, and (3) engineering trustworthy and responsible systems. These thrusts collectively outline a roadmap for elevating text deduplication from a data processing utility to a cornerstone of intelligent and ethical information science.

A. THRUST 1: ADVANCING SEMANTIC INTELLIGENCE AND ROBUSTNESS

A primary research thrust is to imbue deduplication systems with a more profound level of semantic intelligence and resilience, moving beyond current capabilities to achieve a human-like understanding of meaning that is both robust to adversarial manipulation and transparent in its reasoning. The core challenge in semantic detection is moving beyond surface patterns to reliably identify complex paraphrases, implied similarities, and conceptual overlaps that require extensive world knowledge and commonsense reasoning [72], [95], [96]. Even powerful Large Language Models (LLMs) can struggle with this, and their reliability is contingent on their factual consistency and ability to integrate retrieved knowledge [97], [98], [99]. The future direction is to harness the emergent reasoning capabilities of LLMs to detect these nuanced redundancies, such as identifying that a technical bug report and a high-level user complaint describe the same underlying issue [63]. This creates a virtuous cycle

where better deduplication improves LLM training data [2], and in turn, more capable LLMs can identify even more subtle duplicates [100], pushing the boundaries of automated content understanding.

This pursuit of semantic depth is directly challenged by the threat of adversarial attacks, where texts are deliberately manipulated via minimal edits, synonym swapping, or sophisticated paraphrasing to evade detection [18], [101], [102], [103], [104]. As models become more powerful, ensuring their robustness is a critical and unsolved problem. Future research must therefore focus on developing inherently resilient models, perhaps through adversarial training or by designing architectures that are less sensitive to superficial perturbations. The goal is to create systems whose semantic judgments are reliable even in adversarial contexts, a prerequisite for their use in high-stakes applications like plagiarism detection or security.

As these models achieve greater semantic prowess and complexity, their “black box” nature becomes a significant barrier to user trust, error analysis, and accountability [105], [106], [107], [108], [109], [110]. Making their decision-making processes transparent and interpretable is therefore a parallel and equally critical research direction. Future work will leverage the generative power of LLMs themselves, using sophisticated prompting to instruct models not only to identify duplicates but also to articulate the reasoning behind their judgments in natural language [111], [112]. Concurrently, adapting eXplainable AI (XAI) techniques to provide deeper insights into how semantic similarity is computed will be essential [3], [113]. The ultimate objective is to create a new class of deduplication systems that are not just accurate and robust, but also inherently interpretable and trustworthy by design.

B. THRUST 2: SCALING TO NEW FRONTIERS: DATA VOLUME, VELOCITY, AND VARIETY

A second major research thrust is to expand the reach of deduplication to meet the challenges posed by the immense scale, velocity, and variety of modern data. A persistent grand challenge is scaling deduplication to petabyte-scale corpora and high-velocity data streams, which pushes conventional algorithms and architectures to their limits. The quadratic complexity of naive comparisons remains a theoretical barrier, and the computational cost of modern semantic methods [4], [17], alongside immense I/O overhead [15], [16], makes exhaustive analysis infeasible. These issues are amplified in dynamic environments that demand low-latency state management and adaptation to concept drift [114]. Overcoming this requires fundamental innovations in sub-linear approximate similarity search algorithms, adaptive indexing structures for streams, and self-tuning systems that can dynamically optimize the accuracy-resource trade-off [87].

Beyond volume and velocity, real-world utility demands systems that can handle the immense heterogeneity of text

data, including noise from OCR [115], domain-specific jargon [8], [58], and multilingual content [7], [116]. This motivates research into the cross-lingual redundancy challenge, which requires moving beyond simple translation to create universal semantic representations that can identify equivalence between texts in different languages, a task crucial for global information analysis but one that must avoid English-centric biases [117], [118]. An even greater challenge lies on the multi-modal frontier, which demands systems that can bridge the “modality gap” to detect redundancy between text and other data types like images, tables, or video [115], [119]. Successfully developing joint embedding spaces and comparison techniques for such content will transform deduplication from a text-processing task into a holistic content analysis paradigm.

Ultimately, the effectiveness of deduplication cannot be measured in a vacuum; its true value is determined by its impact on downstream applications. False negatives (missed duplicates) can pollute training data and skew analytics [2], while false positives (erroneous merges) can cause the loss of unique information [14]. This highlights a critical gap between isolated algorithmic performance and real-world utility. Future research must therefore shift towards impact-aware evaluation, a paradigm focused on developing methodologies to systematically quantify the sensitivity of downstream tasks to deduplication errors. This could lead to novel frameworks that jointly optimize deduplication strategies and downstream task performance, ensuring that the deduplication process is maximally beneficial for its intended application.

C. THRUST 3: ENGINEERING TRUSTWORTHY AND RESPONSIBLE SYSTEMS

The third, and arguably most critical, research thrust is the commitment to engineering trustworthy and responsible deduplication systems by systematically addressing evaluation, privacy, and fairness. Foundational progress is hampered by the evaluation bottleneck [17], [120], [121], [122] and the lack of rigorous benchmarking standards. Current evaluations often lack reproducibility due to the inherent subjectivity in annotating semantic duplicates and the scarcity of public ground-truth datasets. Furthermore, there is a critical need to assess domain generalization and cross-lingual robustness, as models trained on English web data often fail on specialized domains or low-resource languages. To address methodological biases, benchmarks like PAWS [123] provide adversarial pairs with high lexical overlap but different meanings, challenging models that rely on surface-level features. Similarly, the Massive Text Embedding Benchmark (MTEB) [124] offers a comprehensive framework covering 58 datasets across 112 languages, emphasizing that effective deduplication models must generalize across diverse tasks such as clustering and retrieval. However, PAWS adversarial pairs may not fully represent real-world paraphrase distributions, and MTEB’s coverage remains skewed toward

high-resource languages, limiting conclusions about true multilingual robustness.

Beyond benchmark limitations, the evaluation landscape suffers from a fundamental tension between precision and recall that disproportionately affects downstream applications. As Lee et al. [2] demonstrated, undetected duplicates in training corpora cause models to emit memorized sequences ten times more frequently, directly undermining privacy guarantees and model generalization. Conversely, overly aggressive deduplication risks discarding linguistically diverse expressions of the same concept, particularly in specialized domains where terminology variations carry semantic nuances. This asymmetry necessitates task-specific evaluation frameworks that quantify the downstream cost of false negatives versus false positives, rather than optimizing for generic pairwise accuracy metrics.

This necessitates a concerted effort to develop scalable methods for ground truth creation, such as using synthetic data generation or semi-supervised learning, and to design more holistic, task-aware evaluation metrics that better reflect a system’s utility beyond simple pairwise accuracy [120].

As deduplication is applied to increasingly sensitive data in fields like healthcare and finance, developing advanced privacy-preserving techniques is paramount. Future work must focus on making methods like federated learning, differential privacy, and homomorphic encryption practical and efficient for textual data. This involves designing protocols to train shared similarity models without centralizing raw text [125] and creating noise-injection mechanisms for embeddings that preserve semantic utility while providing verifiable privacy guarantees. The goal is to foster the trust necessary for adopting these technologies in high-stakes, interactive environments where privacy is non-negotiable [126].

Beyond privacy, this thrust encompasses a broader commitment to ethical AI, focusing on fairness, accountability, and the societal impact of deduplication. A key challenge is to detect and mitigate algorithmic bias, ensuring that systems do not unfairly penalize or misrepresent content from underrepresented groups due to skewed training data or biased linguistic assumptions [127]. This requires developing fairness-aware evaluation metrics and collaborating with social scientists to understand the nuanced impacts of deduplication on information diversity [127]. Furthermore, addressing class imbalance is crucial for robust classification performance, as demonstrated in broader machine learning contexts by Ebiaredoh-Mienye et al. [128].

Security research must also address vulnerabilities; for instance, Nkuna et al. [129] discuss integrating security techniques like steganography with neural networks to preserve model efficiency, a concept analogous to ensuring robustness in deduplication systems against adversarial attacks [130]. Accountability will be enhanced through auditable and transparent systems, directly linking to the research on interpretability [131].

Ultimately, this thrust calls for an interdisciplinary approach, engaging ethicists and legal experts to ensure that the algorithmic determination of what constitutes a “copy” versus a novel “transformation” [132] is aligned with societal values and principles of responsible innovation.

D. FUTURE RESEARCH DIRECTIONS

Future work should bridge the gap between deduplication and the broader text understanding landscape. We envision an interdisciplinary approach where deduplication is not merely a preprocessing step but an integral part of an end-to-end pipeline involving *dynamic preprocessing*, *classification-based filtering*, and *prompt-driven reasoning*. This includes developing dynamic deduplication mechanisms that operate during model training, adaptively removing redundant samples on-the-fly to prevent memorization and reduce computational overhead without requiring full dataset reprocessing. Research should focus on developing unified architectures where the insights from semantic deduplication directly inform downstream prompt engineering and retrieval-augmented generation (RAG) systems.

VIII. CONCLUSION

This survey establishes text deduplication as a foundational pillar for modern AI, charting its evolution from foundational syntactic methods like LSH to the deep semantic understanding enabled by LLMs. We argue that its role has transcended mere preprocessing to become a critical determinant of data quality, model performance, and system efficiency across applications ranging from web search to LLM training. The path forward requires tackling key research frontiers: advancing semantic intelligence while ensuring robustness and interpretability; scaling to new data modalities and languages; and engineering responsible systems with formal guarantees for privacy and fairness. Addressing these challenges is essential for realizing the full potential of deduplication as a core technology for an intelligent and trustworthy information ecosystem.

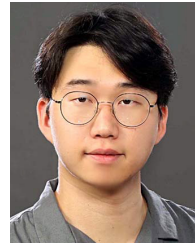
REFERENCES

- [1] F. Xue, Y. Fu, W. Zhou, Z. Zheng, and Y. You, “To repeat or not to repeat: Insights from scaling LLM under token-crisis,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 59304–59322.
- [2] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, “Deduplicating training data makes language models better,” in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 8424–8445.
- [3] D. Hernandez, T. Brown, T. Conerly, N. DasSarma, D. Drain, S. El-Showk, N. Elhage, Z. Hatfield-Dodds, T. Henighan, T. Hume, S. Johnston, B. Mann, C. Olah, C. Olsson, D. Amodei, N. Joseph, J. Kaplan, and S. McCandlish, “Scaling laws and interpretability of learning from repeated data,” 2022, *arXiv:2205.10487*.
- [4] A. Abbas, K. Tirumala, D. Simig, S. Ganguli, and A. S. Morcos, “SemDeDup: Data-efficient learning at web-scale through semantic deduplication,” in *Proc. ICLR Workshop Math. Empirical Understand. Found. Models*, 2023.
- [5] D. Kocetkov, R. Li, L. Ben Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries, “The stack: 3 TB of permissively licensed source code,” 2022, *arXiv:2211.15533*.
- [6] Z. Wang et al., “Data management for large language models: A survey,” 2023, *arXiv:2312.01700v2*.
- [7] M. Abdelhamid, F. Azouaou, and S. Batata, “A survey of plagiarism detection systems: Case of use with English, French and Arabic languages,” 2022, *arXiv:2201.03423*.
- [8] I. Landi, E. Allewa, A. A. Valentine, L. A. Lepow, and A. W. Charney, “Clinical text deduplication practices for efficient pretraining and improved clinical tasks,” 2023, *arXiv:2312.09469*.
- [9] W. Xia, H. Jiang, D. Feng, F. Douglas, P. Shilane, Y. Hua, M. Fu, Y. Zhang, and Y. Zhou, “A comprehensive study of the past, present, and future of data deduplication,” *Proc. IEEE*, vol. 104, no. 9, pp. 1681–1710, Sep. 2016.
- [10] R. Kaur, I. Chana, and J. Bhattacharya, “Data deduplication techniques for efficient cloud storage management: A systematic review,” *J. Supercomput.*, vol. 74, no. 5, pp. 2035–2085, May 2018, doi: 10.1007/s11227-017-2210-8.
- [11] K. Taha, P. D. Yoo, C. Y. Yeun, D. Homouz, and A. Taha, “A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights,” *Comput. Sci. Rev.*, vol. 54, Nov. 2024, Art. no. 100664. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574013724000480>
- [12] B. Hammer, E. Virgili, and F. Bilotta, “Evidence-based literature review: De-duplication a cornerstone for quality,” *World J. Methodol.*, vol. 13, no. 5, pp. 390–398, Dec. 2023.
- [13] S. McKeown and Z. M. Mir, “Considerations for conducting systematic reviews: Evaluating the performance of different methods for de-duplicating references,” *Systematic Rev.*, vol. 10, no. 1, p. 38, Dec. 2021.
- [14] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, “Duplicate record detection: A survey,” *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 1, pp. 1–16, Jan. 2007.
- [15] S. Shenoy, T.-T. Kuo, R. Gabriel, J. McAuley, and C.-N. Hsu, “Deduplication in a massive clinical note dataset,” 2017, *arXiv:1704.05617*.
- [16] U. Niesen, “An information-theoretic analysis of deduplication,” *IEEE Trans. Inf. Theory*, vol. 65, no. 9, pp. 5688–5704, Sep. 2019.
- [17] E. Silcock, L. D’Amico-Wong, J. Yang, and M. Dell, “Noise-robust de-duplication at scale,” National Bureau of Economic Research, Cambridge, MA, USA, Working Paper 30726, 2022.
- [18] M. Y. Zhang, O. Vallis, A. Bumin, T. Vakharia, and E. Bursztein, “RETSim: Resilient and efficient text similarity,” in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 6177–6195.
- [19] S. Weissman, S. Ayhan, J. Bradley, and J. Lin, “Identifying duplicate and contradictory information in Wikipedia,” in *Proc. 15th ACM/IEEE-CS Joint Conf. Digit. Libraries*, Jun. 2015, pp. 57–60.
- [20] M. Richard, L. Giordani, C. Brokate, and J. Liénard, “Unmasking information manipulation: A quantitative approach to detecting copy-pasta, rewording, and translation on social media,” 2023, *arXiv:2312.17338*.
- [21] M. Yan Chen, H. Jiang, and Y. Yang, “Context enhanced short text matching using clickthrough data,” 2022, *arXiv:2203.01849*.
- [22] H. Yanaka, K. Mineshima, P. Martínez-Gómez, and D. Bekki, “Determining semantic textual similarity using natural deduction proofs,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 681–691.
- [23] M. Siino, I. Tinnirello, and M. L. Cascia, “From foundations to GPT in text classification: A comprehensive survey on current approaches and future trends,” *Found. Trends Inf. Retr.*, vol. 19, no. 5, pp. 557–711, Apr. 2025.
- [24] S. Rajaraman, G. Zamzmi, F. Yang, Z. Liang, Z. Xue, and S. Antani, “Semantically redundant training data removal and deep model classification performance: A study with chest X-rays,” *Computerized Med. Imag. Graph.*, vol. 115, Jul. 2024, Art. no. 102379.
- [25] G. Wenzek, M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin, and É. Grave, “CCNet: Extracting high quality monolingual datasets from web crawl data,” in *Proc. 12th Lang. Resour. Eval. Conf.*, 2019, pp. 4003–4012.
- [26] C. Raffel, N. Shazeer, A. P. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2019.
- [27] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 1715–1725.
- [28] S. Sarica and J. Luo, “Stopwords in technical language processing,” *PLoS ONE*, vol. 16, no. 8, Aug. 2021, Art. no. e0254937.

- [29] G. A. Miller, "Wordnet: A lexical database for English," *Commun. ACM*, vol. 38, no. 11, pp. 39–41, 1995.
- [30] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, Mar. 1980.
- [31] M. Siino, I. Tinnirello, and M. La Cascia, "Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on transformers and traditional classifiers," *Inf. Syst.*, vol. 121, Mar. 2024, Art. no. 102342.
- [32] A. Muthithachareon, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *Proc. 18th ACM Symp. Operating Syst. Princ.*, Oct. 2001, pp. 174–187.
- [33] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Commun. ACM*, vol. 18, no. 11, pp. 613–620, Nov. 1975.
- [34] P. Bafna, D. Pramod, and A. Vaidya, "Document clustering: TF-IDF approach," in *Proc. Int. Conf. Electr. Electron., Optim. Techn. (ICEEOT)*, Mar. 2016, pp. 61–66.
- [35] A. Bröder, "On the resemblance and containment of documents," in *Proc. Compress. Complex. Sequences*, 2002, pp. 21–29.
- [36] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Sov. Phys. Doklady*, vol. 163, no. 4, pp. 845–848, 1965.
- [37] W. E. Winkler, "String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage," U.S. Bureau Census, Washington, DC, USA, Tech. Rep. ED 325 505, 1990.
- [38] L. R. Dice, "Measures of the amount of ecologic association between species," *Ecology*, vol. 26, no. 3, pp. 297–302, Jul. 1945.
- [39] G. G. Simpson, "Mammals and the nature of continents," *Amer. J. Sci.*, vol. 241, no. 1, pp. 1–31, Jan. 1943.
- [40] V. Kovatchev, "Paraphrasing, textual entailment, and semantic similarity above word level," Ph.D. dissertation, Departament de Filologia Catalana i Lingüística General, Universitat de Barcelona, Barcelona, Spain, 2020.
- [41] M. Gohsen, M. Hagen, M. Potthast, and B. Stein, "Task-oriented paraphrase analytics," in *Proc. Joint Int. Conf. Comput. Linguistics*, 2024, pp. 15640–15654.
- [42] S. Mujumdar, S. Mehta, H. Patel, and S. Mitra, "Identifying semantically difficult samples to improve text classification," 2023, *arXiv:2302.06155*.
- [43] P. Jaccard, "The distribution of the flora in the Alpine zone.1," *New Phytologist*, vol. 11, no. 2, pp. 37–50, Feb. 1912.
- [44] J. Ciro, D. Galvez, T. Schlippe, and D. Kanter, "LSH methods for data deduplication in a Wikipedia artificial dataset," 2021, *arXiv:2112.11478*.
- [45] A. Broder, S. Glassman, M. S. Manasse, and G. Zweig, "Syntactic clustering of the web," *Comput. Netw. ISDN Syst.*, vol. 29, nos. 8–13, pp. 1157–1166, 1997.
- [46] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. Annu. ACM Symp. Theory Comput.*, May 2002, pp. 380–388.
- [47] B. Gyawali, L. Anastasiou, and P. Knoth, "Deduplication of scholarly documents using locality sensitive hashing and word embeddings," in *Proc. 12th Lang. Resour. Eval. Conf.*, 2020, pp. 901–910.
- [48] A. Shelikhovskii, K. Y. Romanovsky, D. Borovkov, and A. Vasenina, "Interactive duplicate search in software documentation," 2019, *arXiv:1908.08266*.
- [49] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.
- [50] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
- [51] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.
- [52] M. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2018, pp. 2227–2237.
- [53] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [54] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
- [55] J. Devlin and M. Chang, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, Jun. 2018, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [56] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [57] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," 2020, *arXiv:2006.03654*.
- [58] A. K. Dipongkor and K. Moran, "A comparative study of transformer-based neural text representation techniques on bug triaging," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2023, pp. 1012–1023.
- [59] D. Wright and I. Augenstein, "CiteWorth: Cite-worthiness detection for improved scientific document understanding," in *Proc. Findings Assoc. Comput. Linguistics, ACL-IJCNLP*, 2021, pp. 1796–1807.
- [60] K. Kumar Pal, K. Kashiara, U. Anantheswaran, K. C. Kuznia, S. Jagtap, and C. Baral, "Exploring the limits of transfer learning with unified model in the cybersecurity domain," 2023, *arXiv:2302.10346*.
- [61] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, San Francisco, CA, USA, Tech. Rep., 2018.
- [62] I. D. Mienye, N. Jere, G. Obaido, O. O. Ogunraku, E. Esenogho, and C. Modisane, "Large language models: An overview of foundational architectures, recent trends, and a new taxonomy," *Discover Appl. Sci.*, vol. 7, no. 9, p. 1027, Sep. 2025.
- [63] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [64] E. Bandel, Y. Goldberg, and Y. Elazar, "Lexical generalization improves with larger models and longer training," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, 2022, pp. 4398–4410.
- [65] A. Petukhova, J. P. Matos-Carvalho, and N. Fachada, "Text clustering with llm embeddings," 2024, *arXiv:2403.15112*.
- [66] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [67] Y. Ken Chia, G. Chen, L. Anh Tuan, S. Poria, and L. Bing, "Contrastive chain-of-thought prompting," 2023, *arXiv:2311.09277*.
- [68] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. E. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.
- [69] Y. Zha, Y. Yang, R. Li, and Z. Hu, "Text alignment is an efficient unified model for massive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 77942–77968.
- [70] O. K. Oyeyemi and E. Ebenezer, "A syntactically enriched tool enabling dynamic binding to web services," *Int. J. Comput. Sci. Netw. Secur. (IJCSNS)*, vol. 15, no. 4, p. 64, 2015.
- [71] J. Balaji, F. Javed, M. Kejriwal, C. Min, S. Sander, and O. Ozturk, "An ensemble blocking scheme for entity resolution of large and sparse datasets," 2016, *arXiv:1609.06265*.
- [72] K. Tirumala, D. Simig, A. Aghajanyan, and A. S. Morcos, "D4: Improving LLM pretraining via document de-duplication and diversification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53983–53995.
- [73] L. Zhou, J. Chen, A. Das, H. Min, L. Yu, M. Zhao, and J. Zou, "Serving deep learning models with deduplication from relational databases," *Proc. VLDB Endowment*, vol. 15, no. 10, pp. 2230–2243, Jun. 2022.
- [74] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.
- [75] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, pp. 58–75, Apr. 2005.
- [76] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm," in *Proc. Discrete Math. Theor. Comput. Sci.*, Jan. 2007, pp. 137–156.
- [77] G. Hu, Y. Zhang, S. Rigo, and T. D. Nguyen, "Similarity driven approximation for text analytics," 2019, *arXiv:1910.07144*.
- [78] A. Mahadevan, M. Mathioudakis, E. Mäkelä, and M. Tolonen, "Optimizing a data science system for text reuse analysis," 2024, *arXiv:2401.07290*.

- [79] G. Alexiou, G. Papastefanatos, V. Stamatopoulos, G. Koutrika, and N. Koziris, "QueryER: A framework for fast analysis-aware deduplication over dirty data," 2022, *arXiv:2202.01546*.
- [80] Z. Wang, X. Li, R. Xia, and P. Liu, "MathPile: A billion-token-scale pretraining corpus for math," 2023, *arXiv:2312.17120*.
- [81] J. Dean and S. Ghemawat, "Mapreduce: Simplified data processing on large clusters," in *Proc. OsdI*, vol. 4, 2004, p. 5.
- [82] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Syst. Design Implement.*, Apr. 2012, pp. 15–28.
- [83] A. Khan, C.-G. Lee, P. Hamandawana, S. Park, and Y. Kim, "A robust fault-tolerant and scalable cluster-wide deduplication for shared-nothing storage systems," in *Proc. IEEE 26th Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst. (MASCOTS)*, Sep. 2018, pp. 87–93.
- [84] U. Manber, "Finding similar files in a large file system," in *Proc. USENIX Winter Tech. Conf. (USENIX Winter Tech. Conference)*, 1994, p. 2.
- [85] P. Gopalan and J. Radhakrishnan, "Finding duplicates in a data stream," in *Proc. 20th Annu. ACM-SIAM Symp. Discrete algorithms*, 2009, pp. 402–411.
- [86] F. Retkowsky and A. Waibel, "From text segmentation to smart chaptering: A novel benchmark for structuring video transcriptions," 2024, *arXiv:2402.17633*.
- [87] K. Morton, H. Hajishirzi, M. Balazinska, and D. Grossman, "View-driven deduplication with active learning," 2016, *arXiv:1606.05708*.
- [88] N. Sahlab, H. Kahoul, N. Jazdi, and M. Weyrich, "A knowledge graph-based method for automating systematic literature reviews," *Proc. Comput. Sci.*, vol. 207, pp. 2814–2822, Jan. 2022.
- [89] N. Meuschke, V. Stange, M. Schubotz, M. Kramer, and B. Gipp, "Improving academic plagiarism detection for STEM documents by analyzing mathematical content and citations," in *Proc. ACM/IEEE Joint Conf. Digit. Libraries (JCDL)*, Jun. 2019, pp. 120–129.
- [90] D. V. Luciv, D. V. Koznov, G. A. Chernishev, A. N. Terekhov, K. Y. Romanovsky, and D. A. Grigoriev, "Detecting near duplicates in software documentation," *Program. Comput. Softw.*, vol. 44, no. 5, pp. 335–343, Sep. 2018.
- [91] Y. Niu, C. Qiao, H. Li, and M. Huang, "Word embedding based edit distance," 2018, *arXiv:1810.10752*.
- [92] I. Malkiel, D. Ginzburg, O. Barkan, A. Caciularu, Y. Weill, and N. Koenigstein, "Metricbert: Text representation learning via self-supervised triplet training," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 1–5.
- [93] G. S. Manku, A. Jain, and A. Das Sarma, "Detecting near-duplicates for web crawling," in *Proc. 16th Int. Conf. World Wide Web*, May 2007, pp. 141–150.
- [94] A. T. Nguyen, T. T. Nguyen, T. N. Nguyen, D. Lo, and C. Sun, "Duplicate bug report detection with a combination of information retrieval and topic modeling," in *Proc. 27th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Sep. 2012, pp. 70–79.
- [95] H. Gilbert, M. Sandborn, D. C. Schmidt, J. Spencer-Smith, and J. White, "Semantic compression with large language models," in *Proc. 10th Int. Conf. Social Netw. Anal.*, 2023, pp. 1–8.
- [96] L. Hagström, D. Saynova, T. Norlund, M. Johansson, and R. Johansson, "The effect of scaling, retrieval augmentation and form on the factual consistency of language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 5457–5476.
- [97] S. Setty, H. Thakkar, A. Lee, E. Chung, and N. Vidra, "Improving retrieval for RAG based question answering models on financial documents," 2024, *arXiv:2404.07221*.
- [98] S. Xu, L. Pang, M. Yu, F. Meng, H. Shen, X. Cheng, and J. Zhou, "Unsupervised information refinement training of large language models for retrieval-augmented generation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 133–145.
- [99] S. Das, L. Jin, L. Song, H. Mi, B. Peng, and Y. Dong, "Entropy guided extrapolative decoding to improve factuality in large language models," in *Proc. 31st Int. Conf. Comput. Linguistics*, 2024, pp. 6589–6600.
- [100] A. Neelakantan et al., "Text and code embeddings by contrastive pre-training," 2022, *arXiv:2201.10005*.
- [101] M. Almagro, E. Almazán, D. Ortego, and D. Jiménez, "LEA: Improving sentence similarity robustness to typos using lexical attention bias," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 36–46.
- [102] X. Chen, G. Long, C. Tao, M. Li, X. Gao, C. Zhang, and X. Zhang, "Improving the robustness of summarization systems with dual augmentation," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 6846–6857.
- [103] X. He, X. Shen, Z. Chen, M. Backes, and Y. Zhang, "MGTBench: Benchmarking machine-generated text detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 2251–2265.
- [104] W. Qu, D. Yin, Z. He, W. Zou, T. Tao, J. Jia, and J. Zhang, "Provably robust multi-bit watermarking for ai-generated text via error correction code," 2024, *arXiv:2401.16820*.
- [105] J. Choi, E. Jung, S. Lim, and W. Rhee, "Finding inverse document frequency information in BERT," 2022, *arXiv:2202.12191*.
- [106] W. M. Lim and H. Tayyar Madabushi, "UoB at SemEval-2020 task 12: Boosting BERT with corpus level information," in *Proc. 14th Workshop Semantic Eval.*, 2020, pp. 2216–2221.
- [107] I. Malkiel, D. Ginzburg, O. Barkan, A. Caciularu, J. Weill, and N. Koenigstein, "Interpreting BERT-based text similarity via activation and saliency maps," in *Proc. ACM Web Conf.*, 2022, pp. 3259–3268.
- [108] Z. Wang, Y. Zhang, and H. Wu, "Structural-aware sentence similarity with recursive optimal transport," 2020, *arXiv:2002.00745*.
- [109] Z. Wang, J. Dou, and Z. Yon, "Unsupervised sentence textual similarity with compositional phrase semantics," in *Proc. 29th Int. Conf. Comput. Linguistics*, 2022, pp. 4976–4995.
- [110] B. Joshi, N. Shah, F. Barbieri, and L. Neves, "The devil is in the details: Evaluating limitations of transformer-based methods for granular tasks," in *Proc. 28th Int. Conf. Comput. Linguistics*, 2020, pp. 3652–3659.
- [111] L. Ma, N. Thakurdesai, J. Chen, J. Xu, E. Körpeoglu, S. Kumar, and K. Achan, "LLMs with user-defined prompts as generic data operators for reliable data processing," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2023, pp. 3144–3148.
- [112] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Large language models as data preprocessors," 2023, *arXiv:2308.16361*.
- [113] Y. Dong, C. Bhagavatula, X. Lu, J. D. Hwang, A. Bosselut, J. C. K. Cheung, and Y. Choi, "On-the-fly attention modulation for neural generation," in *Proc. Findings Assoc. Comput. Linguistics, ACL-IJCNLP*, 2021, pp. 1261–1274.
- [114] I. Lourentzou, K. Manghnani, and C. X. Zhai, "Adapting sequence to sequence models for text normalization in social media," in *Proc. Int. AAAI Conf. Web social media*, vol. 13, 2019, pp. 335–345.
- [115] J. Li, W. Gu, K. Ota, and S. Hasegawa, "Object recognition from scientific document based on compartment and text blocks refinement framework," *Social Netw. Comput. Sci.*, vol. 5, no. 7, Aug. 2024.
- [116] Y. Wang, J. Mansurov, P. M. Ivanov, J. Su, A. Shelmanov, A. Tsvigun, C. Whitehouse, O. M. Afzal, T. M. Mahmoud, A. F. Aji, P. Nakov, A. A. Fikri, H. Nizar, G. Iryna, and N. Preslav, "M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics*, 2023, pp. 1369–1407.
- [117] L. Lucy, S. Gururangan, L. Soldaini, E. Strubell, D. Bamman, L. Klein, and J. Dodge, "AboutMe: Using self-descriptions in web-pages to document the effects of english pretraining data filters," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
- [118] Y. Lyu, Z. Li, S. Niu, F. Xiong, B. Tang, W. Wang, H. Wu, H. Liu, T. Xu, and E. Chen, "CRUD-RAG: A comprehensive Chinese benchmark for retrieval-augmented generation of large language models," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–32, Mar. 2025.
- [119] S. Kulkarni, S. Govilkar, and D. Amin, "Text similarity from image contents using statistical and semantic analysis techniques," in *Proc. Artif. Intell., NLP, Data Sci. Cloud Comput. Technol.*, Aug. 2023, pp. 37–46.
- [120] O. Binette, Y. Baek, S. Engineer, C. Jones, A. Dasylyva, and J. P. Reiter, "How to evaluate entity resolution systems: An entity-centric framework with application to inventor name disambiguation," 2024, *arXiv:2404.05622*.
- [121] D. Brook Weiss, P. Roit, O. Ernst, and I. Dagan, "Extending multi-text sentence fusion resources via pyramid annotations," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2022, pp. 1854–1860.
- [122] D. Brook Weiss, P. Roit, A. Klein, O. Ernst, and I. Dagan, "QA-align: Representing cross-text content overlap by aligning question-answer propositions," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 9879–9894.

- [123] Y. Zhang, J. Baldridge, and L. He, "PAWS: Paraphrase adversaries from word scrambling," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 1298–1308.
- [124] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proc. 17th Conf. Eur. Chapter Assoc. Comput. Linguistics*, 2023, pp. 2014–2037.
- [125] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2016, pp. 1273–1282.
- [126] D. Vatsalan, P. Christen, and V. S. Verykios, "A taxonomy of privacy-preserving record linkage techniques," *Inf. Syst.*, vol. 38, no. 6, pp. 946–969, Sep. 2013.
- [127] S. L. Blodgett, S. Barocas, H. Daumé, and H. Wallach, "Language (technology) is power: A critical survey of 'bia' in NLP," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 5454–5476.
- [128] S. A. Ebiaredoh-Mienye, "Improved machine learning methods for classification of imbalanced data," Dept. Elect. Electron. Eng., Univ. Johannesburg, Tech. Rep., 2021.
- [129] M. C. Nkuna, A. Ali, and E. Esenogho, "Integrating image steganography techniques into an existing efficient image recognition artificial neural network for optimal performance," in *Proc. IEEE PES/IAS PowerAfrica*, Oct. 2024, pp. 1–5.
- [130] M. Alzantot, Y. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, "Generating natural language adversarial examples," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2890–2896.
- [131] P. Hartel, E. Haspels, M. van Staaldin, and O. Teixeira, "Dark-Diff: Explainable web page similarity of TOR onion sites," 2023, *arXiv:2308.12134*.
- [132] A. Kolchinsky and B. Corominas-Murtra, "Decomposing information into copying versus transformation," *J. Roy. Soc. Interface*, vol. 17, no. 162, Jan. 2020, Art. no. 20190623.



JEONGMIN KIM is currently pursuing the B.S. degree in computer science and engineering with Kangwon National University, Chuncheon, Republic of Korea. As an Undergraduate Researcher with the Big Data Laboratory, he has worked on projects related to big data and machine learning under the supervision of Yeonsu Park. His research interests include big data processing, text deduplication, and advanced code intelligence.



YEONSU PARK received the B.S. degree in software engineering from Sungkyunkwan University, Suwon, Republic of Korea, in 2017, and the Ph.D. degree in computer science and engineering from POSTECH, Pohang, Republic of Korea, in 2024. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Kangwon National University, Chuncheon, Republic of Korea. Before that, he was a Postdoctoral Research Scientist with the Graduate School of Artificial Intelligence (GSAI), POSTECH. His research interests include big data processing, database systems, query processing, and query optimization.

...